Project Report

A multi-metric candidate discovery method using multi-class ensemble selection for multimedia routing in software defined networks was evaluated on both GEANT2 and fat-tree topologies.

Base Research Paper :

A.D. İpek et al., "AirSdn: Ai-Based Routing In Software Defined Networks For Multimedia Traffic Transmission," *Computer Communications*, Vol. 240, 2025.

Submitted by

Anshul Marathe (106124019)

Rahul Shrivastava (106124093)

Sanskar Sharma (106124109)

Multi-Metric Candidate Discovery with Multi-Class Ensemble Selection for Multimedia Routing in SDN: An Evaluation on GEANT2 and Fat-Tree Topologies

Anshul Marathe, Rahul Shrivastava, Sanskar Sharma

***Abstract*—The video's quality will be reduced if a Software Defined Network (SDN) Controller sends the video through a congested link. In order to avoid this problem, the authors of the original paper, "AIRSDN," probed the five shortest possible paths based upon hop count and selected the path that was labeled as being lightly loaded using a Binary Logistic Regression Model. However, since all of the potential paths were chosen based upon hop count, some of those paths could have crossed the same congested link and the two labels used in the model did not differentiate between a path at 40% utilization versus a path at 5%. Therefore, we replaced both aspects of the methodology. The first step involved running Yen's Algorithm with a normalized cost of delay, jitter, inverse available bandwidth and loss and selecting the five lowest-cost paths. Then, in the second phase, we probed each of the five candidates for 11 different characteristics; after doing so, we used a soft-voting ensemble of XGBoost, Random Forest, Gradient Boosting and Support Vector Machines (SVM) to classify each of the five paths into one of four levels of congestion in a single batched operation. To create our system, we utilized Mininet and Floodlight and evaluated it on two networks: a 24-node GEANT2 network and a 20-node K=4 Fat Tree network. For each network, we collected 500 (GEANT2) and 896 (fat-tree) live-measured training paths and created five different scenarios per network that forced the hop-count path to extremely high levels of utilization. When evaluating our approach on the GEANT2 network, we found that our method always chose the least congested reachable path for each scenario and provided us with a 66-second H.264 video without any losses, at 38.4 decibels Peak Signal-to-Noise Ratio (PSNR), and with an average Round Trip Time (RTT) of 157 milliseconds. Conversely, the hop-count-based candidate method lost 2.1 percent of the video and had a PSNR value of 31.4 decibels and an average RTT of 532 milliseconds. Additionally, we were able to reduce the amount of time required to make a decision by 41 percent from 217 seconds to 128 seconds. Finally, on the fat tree network every load-aware method (including the hop-count-based candidate method) already selected the best available path, because its few alternative paths were already among the candidates, which left no opportunity for further improvement.**

***Index Terms*—software-defined networking, QoS routing, multi-class classification, ensemble learning, multimedia streaming, k-shortest paths**

I. Introduction

Real-time video and other forms of media now make up most of the traffic in today's research and data center networks; however, these are also the most vulnerable to network congestion. A few percent of lost packets will corrupt several consecutive frames in an H.264 video stream. Software Defined Networking (SDN) separates the control plane from the forwarding plane. The control plane includes a single logical controller which sees the whole network. From this perspective, the controller can make decisions about where to send or distribute each flow depending on what is happening at any given time across all of the links rather than relying on pre-defined metrics. Both Quality-of-Experience (QOE)-based SDNs and Load-Aware SDNs rely on this concept [2], [6], [7].

Controllers continue to rely on hop count-based routing and/or single link metrics like delay when determining paths for data transmission. Both of these methods fail to consider how much traffic will be transmitted across all available paths and also cannot adequately represent the trade off between choosing a shorter path and an idle one. There are three main categories of learning-based approaches that have been created: predicting future traffic patterns and then utilizing those predictions to select routes [3]; learning a load-balancing policy from past data [5]; and evaluating multiple options for paths currently available and choosing the best option from those choices (this method was employed by AIRSDN [1]).

There are two main advantages to using AIRSDN: First, it is both simple to use and to evaluate. Second, it probes the five hop-count shortest path(s) in a straightforward manner, and then classifies them as either "low" or "high" using logistic regression based upon the quantity of traffic passing through each link. Finally, AIRSDN always chooses the "low." However, there are also two significant disadvantages to AIRSDN's design. One disadvantage is that all potential candidates for selection are evaluated solely on the basis of their hop count. If congestion exists over the shorter paths, then every candidate will be congested and regardless of how well the classification algorithm performs, it will only choose the best among bad choices. A second disadvantage is that because AIRSDN has only two possible classifications ("low" and "high"), it does not provide sufficient granularity to differentiate among several different "low" options.

In this research, we have eliminated both constraints.

In stage 1 of our approach, we create potential routes based upon a composite cost which considers the amount of bandwidth available at any given time, along with the delay, jitter and loss associated with using that particular route. As such, if there exists an alternative route that avoids a congested area then it will be included within the pool of possible options.

In addition to the above-mentioned methodology for improving the decision-making process for routing decisions on congested links, this paper also describes an evaluation of the proposed method. The second phase of our methodology was implemented using a soft voting ensemble which classified every possible choice into one of the four categories of congestion. Once all choices were classified, they were evaluated together in a single batch process to determine which choice would be made. Our overall performance was evaluated based upon a real-time H.264 video feed transmitted over two very different types of networks: GEANT2 (a large-scale meshed wide area network with a central hub) and a fat tree data center fabric (four equal-cost shortest path routes). In addition to comparing our results to those produced using AIRSDN's baseline method, we compared our results to those produced using hop count routing, latency-based routing, and bandwidth-based routing. Lastly, we measured how long it took to complete each individual decision-making step.

II. Literature Survey

QoE-driven content delivery. Kük & Erel-Özçevik [2], apply software-defined controls to a content delivery network (CDN) in mobile networks for the purpose of optimizing the quality of experience (QoE) experienced by the end-users. Although Kük & Erel-Özçevik [2] have similar objectives as those being pursued in this project regarding the protection of multimedia quality via centralized control; however, they make their decisions based upon how content will be delivered within a CDN and not on which one or multiple measured network path(s) a single flow should be routed. Similarly, Bakonyi et al. [4], utilize classification to guide load balancing in content delivery networks. Similar to AIRSDN and Stage 2 of this work, a classifier is used to provide input to the decision-making process; however, unlike AIRSDN and Stage 2 of this work, Bakonyi et al. [4]'s decision relates to where content should be served and not how it should be routed.

Prediction-driven path selection. Cui & Xu [3], used a neural network to predict traffic so they could make choices regarding how to route their cloud computing traffic based upon predicted loads. In contrast, this research measures all potential routes prior to selecting one; thus eliminating the dependency on accurate predictions but increasing the amount of time required to probe.

Load balancing in SDN. Babayigit & Ulu [5], utilized Deep Learning for Load Balancing in SDN-Based Data Center Networks. In addition, Mohamed et al. [7] proposed a Server Based Load Balancing Approach Using SDN; they demonstrated how the Controller's View Could Be Used To Distribute Demand Across Servers. Finally, Al-Mimi et al. [6] surveyed Artificial Intelligence Based Load Balancing Schemes in SDN and therefore defined the Design Space Wherein Learned Selectors Such As AIRSDN And This Work Reside.

The base paper. AIRSDN [1] is a measurement-based approach as opposed to prediction-based; i.e., for every multimedia flow it finds the five hop-count shortest paths using probing, then computes 19 path metrics/attributes, uses logistic regression to assign a label to each path, and finally sets up a low-traffic path on NSFNET using Mininet/Floodlight. AIRSDN reported an average RTT of 354 ms and a PSNR of 37.3 dB compared to 2,063 ms and 21.5 dB when using hop count based routing. Table I presents a comparison between AIRSDN and other related methods along with our proposed methodology.

III. Base Paper Analysis and Research Gap

Objective and approach. AIRSDN [1] sends every multimedia stream along the path that has been determined to be the least congested of a very limited number of possible paths based on measurements taken prior to sending the stream. Once a new stream begins to send data, AIRSDN calculates the five shortest-hop-count paths connecting the two end-points using Yen's Algorithm [8]. Then it sends test traffic across each one of those paths one at a time while measuring 19 different characteristics related to Ping, Iperf3, and Flow Statistics. An off-line-trained Logistic Regression Model will label each of these paths either Low-Traffic or High-Traffic. Finally, AIRSDN will create static OpenFlow rules to forward the stream down the first path labeled Low-Traffic.

Contributions. A small number of potential routes can be measured and classified to allow for video routing around congested areas; this has been demonstrated using NSFNET where the average RTT was reduced from 2,063 milliseconds (based upon hop count) to 354 milliseconds while increasing PSNR from 21.5 to 37.3 decibels. Additionally, the pipeline is cost effective when making decisions, easily understood, and will operate without modification on an existing controller.

Limitations. Three limitations follow from the design.

1) *Hop-count candidates.* Typically, there are at least five of the shortest paths; however, if the congested links exist along each one then no matter which path you choose it will be congested and therefore the classifier has to select the best option available.
2) *Binary labels.* Two paths with low load will be unable to be ranked using binary labels ("low" vs. "high"), despite the fact that a 4.7 Mbit/s video can fill up a 10 Mbit/s link which has been previously filled to 50%.
3) *One topology.* The method was tested only on the 14 node NSFNET; therefore, it is unclear how this method would perform on other types of network topologies.

Research gap addressed. In this research, we will use

TABLE I
COMPARISON OF RELATED WORK TO PROPOSED METHOD

Reference	Problem addressed	Approach / technique	Key contribution	Limitation
SD-CDN [2]	QoE of content delivery in mobile networks	software-defined control of a CDN	SDN-based CDN design aimed at QoE	content-delivery decision, not per-flow path choice
Cui & Xu [3]	path selection for cloud traffic	neural network traffic prediction drives the path choice	prediction-based path selection	relies on forecast load, not on measuring the candidate paths
Bakonyi et al. [4]	load balancing in CDNs	classification-based balancing	a classifier informs the balancing decision	balances where content is served, not how a flow is routed
Babayigit & Ulu [5]	load balancing in SDN data centers	deep learning	learned load balancing for SDN data center networks	data center balancing, not multimedia path selection
Al-Mimi et al. [6]	AI-based load balancing in SDN	survey	overview of AI load-balancing schemes for SDN	survey; no evaluated scheme
Mohamed et al. [7]	server load balancing	SDN-based server-side balancing	SDN controller spreads demand across servers	server selection, not network path selection
AIRSDN [1] (base)	multimedia routing in SDN	probe 5 hop-count paths, Logistic Regression (2 classes)	measured, low-overhead ML routing; 354 ms RTT on NSFNET	hop-count candidates only; binary labels; one topology
Proposed	multimedia routing under concentrated congestion	composite-cost top-5 + 4-class soft-voting ensemble, batched	finds detours outside the congested region; graded classes; evaluated on two topologies	probing time per decision; no gain where no better path exists

the same "measure then classify" framework as AIRSDN; however, we will be measuring and evaluating things differently than in the base paper. In addition to using the original contribution of the base paper which was the probing plus classification selector based upon hop count candidate selection, we have made three additional contributions: first, we will develop a multiple metric stage 1 that selects candidates from outside of the congested area; secondly, we will create four congestion classifications using a soft voting ensemble where each candidate will receive a score at once during a single batched evaluation; lastly, we will evaluate our model on two very different types of networks (GEANT2 and Fat Tree), while also performing a per step overhead analysis.

IV. Problem Statement and Objectives

Problem. Find and execute a path for a multimedia transmission between a Sender and a Receiver in a Software-Defined Network (SDN) with active background traffic and a congested default path, which minimizes the packet loss and latency of the transmission, based solely on the measurements available to the controller before the flow is initiated, and such that the cost of the decision is consistent with per-flow routing.

Objectives.

1) Design a two stage router where the candidate list contains routes which do not pass through the congested area. Then determine if this router will select the least congested route available in every scenario.
2) Create an ensemble of four different classes using a soft voting method for each network topology used. The ensemble should have greater than 95% accuracy on both the test set and the Macro F1 score.
3) Compare the performance of your model against the AIRSDN style baseline and Hop Count, Latency, Bandwidth based routing methods on throughput, packet loss, RTT, jitter, PSNR and SSIM, using GEANT2 and a Fat Tree ($k = 4$).
4) Measure how long it takes to make decisions at each stage and demonstrate that Stage 2 inferences are made simultaneously across all candidates.

Scope & Expected Outcome. This research will use emulated 10 megabit per second (Mbit/s) networks having a controlled four-class background traffic. The goal for this study is that when there is an available non-congested route then all data should be delivered without any losses; however, it also requires that the time required to make decisions about routing be no greater than those made by the base-line method.

V. Proposed Methodology

A. Overview of the Proposed Approach

The router has the option to make one routing decision per multimedia stream at two distinct times (refer to Figure 1). Before it can make its decision, the Controller collects information about every link. At Stage 1, the five possible paths are determined using a composite cost function that is calculated on the basis of the graph created from the data. In Stage 2, eleven attributes of each possible path are extracted and then presented to an Ensemble Model. Once the attribute values have been calculated, the path with the lowest predicted congestion class is selected and forwarded to Floodlight for execution.

B. Architecture

The system has five modules on top of Mininet, Open vSwitch and the Floodlight controller:

- *Topology and traffic.* The creation of the network will

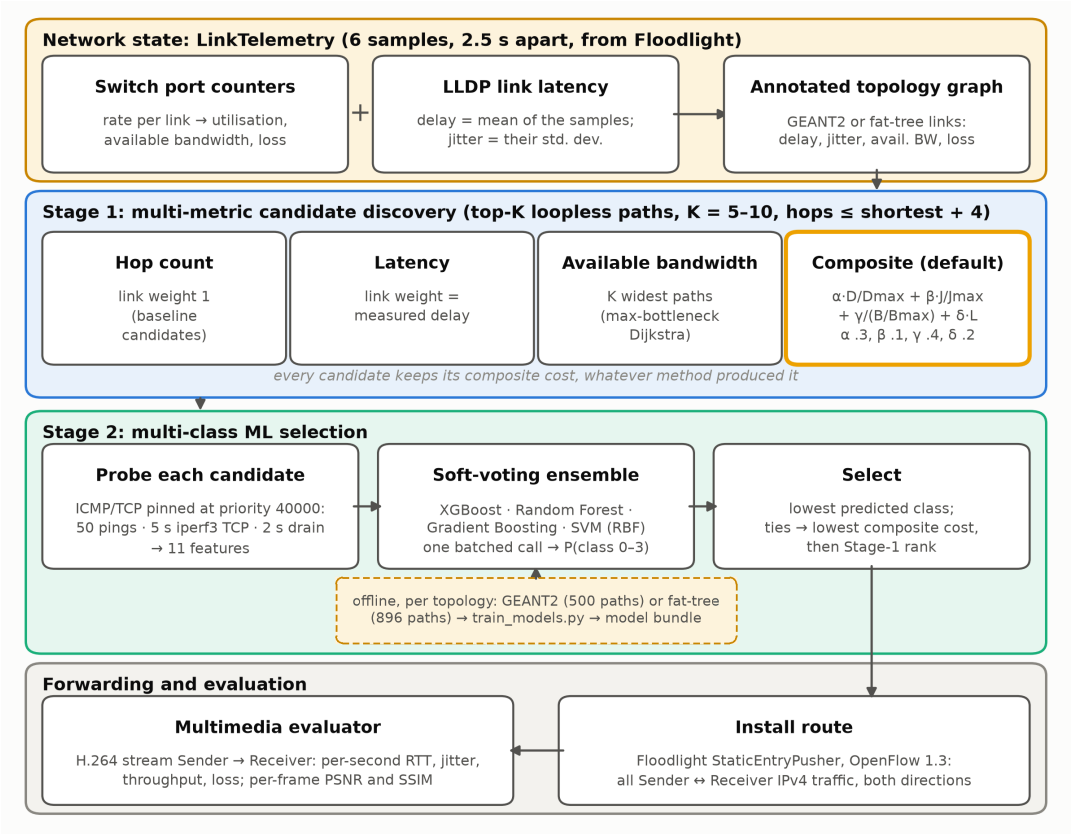


Fig. 1. The architecture of the proposed two-stage router. Candidate paths can be chosen using any of four different types of metrics (composite is the default). Stage 2 will probe these paths and pick the one with the lowest congestion class, and then install that path as a static OpenFlow entry.

result in the generation of four types of background traffic. Each link will have one IPERF3 UDP flow per direction with a bandwidth that has been randomly selected from one of the four possible load bands (Extremely low < 0.25; low < 0.5; high < 0.8; extremely high $<= 1.2$) of the total available capacity. The "ground truth" for each path is determined by the type of load on the link which carries the greatest amount of traffic.

- *Telemetry.* The floodlight is sampled six times with an interval of 2.5 seconds between each sample. From these samples, per-link metrics including delay and jitter (mean and standard deviation) for LLDP latency, available bandwidth (total capacity minus the maximum utilization on either end) and packet loss (using port counter data from both ends) can be calculated.
- *Stage 1: Path Discovery.* The first stage involves using a modified Hop-Limited Version of Yen's Algorithm [8] that takes into account hop-count, delay, maximum bandwidth available along a path, or any combination of these factors.
- *Stage 2: Selection.* In this stage, we will extract features from each node/edge; probe for potential paths; utilize multiple algorithms together as an Ensemble; and make decisions regarding which path(s) should be selected.
- *Evaluation.* During evaluation, video will be transmitted while collecting QoS data at one-second intervals and assigning scores to each individual frame based on the quality of that frame.

C. Proposed Algorithm

Composite cost. For link uv with delay D_{uv}, jitter J_{uv}, available bandwidth B_{uv} and loss fraction L_{uv}, normalised by the maximum over all links,

$$c_{uv} = \alpha \frac{D_{uv}}{D_{\max}} + \beta \frac{J_{uv}}{J_{\max}} + \frac{\gamma}{\max(B_{uv}/B_{\max},\, 10^{-3})} + \delta L_{uv}, \tag{1}$$

where α=0.3, β=0.1, γ=0.4 and δ=0.2. Since the inverse bandwidth term will increase rapidly when a link becomes full, the cheapest paths avoid loaded links even when they are longer. A path's cost is the sum of its link costs, and Yen's algorithm returns the five cheapest loopless paths of at most the shortest hop distance plus four hops. Every candidate also carries its composite cost into Stage 2.

Features. Each probe has ICMP and TCP traffic prioritized above that of the route, therefore no disturbance will occur to the video during each probe. The 11 characteristics include: Mean, Min, Max & Deviation of RTT; Loss of 50 Pings; Throughput; Bandwidth Utilization; Retransmissions; Hop Count; Link Utilization Max & Mean (from Telemetry Snapshot).

Ensemble. A total of four different types of classifiers were developed for each type of network topology; these in-

cluded XGBoost, Random Forest, Gradient Boosting, and an RBF Support Vector Machine with Platt-Calibrated Probabilities. The deployed model is their soft vote: for a feature vector $\mathbf{x}$,

$$\hat{y}(\mathbf{x}) = \arg \max_{c \in \{0,1,2,3\}} \frac{1}{4} \sum_{m=1}^{4} P_m(c \mid \mathbf{x}). \quad (2)$$

The candidate with the lowest predicted class is selected. When there are multiple candidates that have the same predicted class, the one with the smallest composite cost is chosen first. If two or more candidates still have the same composite cost after this selection process has taken place, then they are ranked according to their position in Stage 1.

Why an ensemble rather than a single classifier. There are three characteristics of this problem which make a soft voting ensemble preferable to each individual member.

1) *Variance reduction and generalisation.* A single non-linear model (i.e., a decision tree) has been shown to have a high degree of sensitivity to even minor changes in input values; therefore, when using live probe data collected during periods of heavy "bursty" background traffic, it is possible for the same path to generate significantly different feature vectors at each successive decision point. The use of averaging across the probability outputs of four separate models that were independently trained will help reduce the variance associated with each individual model's placement of a boundary.

2) *Complementary decision boundaries.* Each member has a unique set of inductive biases. As an example, the RBF SVM will create smooth and distance based decision boundaries across all of the continuous telemetry features (RTT, etc.) and their deviations; whereas the three types of tree-based learners (XGBoost, Gradient Boosting and Random Forest) will identify step-like threshold points where one feature transitions from one level of utilization to another. In (2), soft voting is used to combine the calibrated probability estimates $P_m(c|\mathbf{x})$ produced by each type of learner rather than selecting just one.

3) *Reliable multi-class output.* Averaging the probability will decrease the effect of a single member's confidence in making an incorrect choice about an abnormal candidate path; thus, a wrong class will be selected only if it has received enough probability from all other members. The reason this is significant is due to how the selection process utilizes the predicted class for each candidate.

There were no members of this group that demonstrated optimal performance on both network architectures (GEANT2 & Fat Tree) based on the empirical data collected from Table IV. Random Forest, XG Boost, and Gradient Boosting were equally capable of performing on GEANT2. However, Random Forest was the most successful performer on the Fat Tree. Additionally, Support Vector Machine (SVM) was the poorest performer on both networks. On each topology, the ensemble either performed as well as or better than three of its four members. Additionally, the ensemble accomplished this without using a different model for each topology. With respect to 10x5 fold cross-validation on the GEANT2 data set, the ensemble generated a macro F-1 score of 0.982 $\pm$ 0.014 with a mean log loss of 0.094 while Gradient Boosting alone generated a mean log loss of 0.202. This indicates that the use of averaging generates better calibrated probability estimates than those generated by individual models.

Why four traffic classes? Background traffic, the dataset labels and the classifier all use four load bands rather than AIRSDN's two, for two reasons.

1) *Ranking candidates that are all "low".* A binary label will not be able to rank candidates which have been placed on either side of its threshold. Both paths with a bottle-neck carrying 45% of their 10 Mbit/s capacity and another with 5% are considered "low"; however, when you add the 4.7 Mbit/s video, the first path increases to approximately 92% utilization and begins building queues and losing frames while the second increases to approximately 52%. Therefore there are four different levels of classification for these two types of candidates and therefore your selection rule should select the candidate with less load.

2) *Non-linear queueing behaviour.* Delay does not increase linearly based upon the amount of traffic being processed. For example, when using an M/M/1 queue where there is a service rate of μ and utilization of ρ, then the average time spent in the system will be $T = 1/[\mu(1-\rho)]$. Compared to an idle link, the average time spent in the system will be approximately 1.3x larger when ρ=0.25; approximately 2x larger when ρ=0.5; approximately 5x larger when ρ=0.8; and will continue to grow indefinitely as ρ approaches 1. At this point, the system has reached its maximum capacity and the queue is filled with packets that cannot enter the system. Consequently, there are four distinct levels of performance (Extremely Low, Low, High, and Extremely High), each having similar properties concerning both delay and packet loss. Delays and losses are relatively stable over the range of Extremely Low and Low performance categories. However, delays and losses do increase slightly within the High performance category, and they increase significantly within the Extremely High performance category. Similar trends were observed in all of the real-time data sets we collected for our studies (Section VII-C).

TABLE II
MODIFICATIONS TO AIRSDN

Aspect	AIRSDN [1]	Proposed
Candidates	5 hop-count shortest paths	5 cheapest paths under the composite cost (1); detours allowed up to +4 hops
Labels	2 (low / high)	4 (Extremely Low ... Extremely High)
Model	Logistic Regression	soft vote of XGBoost, RF, GB, SVM (2)
Inference	per candidate	one batched call for all K
Features	19 (incl. flow statistics)	11 (ping, iperf3, hop count, link utilisation)
Tie-break	Stage 1 rank	composite cost, then rank
Topologies	NSFNET	GEANT2 and $k = 4$ fat-tree, one model each

D. Modifications to the Base Paper

Table II shows the changes made to the system; why those changes were made; and how they resolve each of the three limitations identified in section III. Composite candidate solutions address Limitation 1 as they allow for the inclusion of a very low-cost path (all links are idle) in the set of possible paths; therefore, composite candidates can include these types of detours. Candidate classes that rank lightly loaded candidates provide a solution for Limitation 2. Batching allows for the inference cost to remain relatively constant as compared to K. A single model per topology provides a solution for Limitation 3 as it does not require one model to make predictions on RTT values outside of its training range.

E. Procedure for performing the work

- *Offline:* A labeled path data set was collected from the live emulated topology (section VI), then used to train the four models along with their ensemble.
- *Telemetry:* Once a flow has arrived at the system, a link snapshot (approximately 19 seconds) will be taken.
- *Stage 1:* The graph will be annotated and the top 5 composite cost paths will be calculated.
- *Probe:* Each candidate path will have ICMP/TCP pinned to it. Fifty ping requests will be sent to the destination node followed by a 5 second iperf3 transfer. After waiting two seconds for the queue to empty out, the pin will be removed.
- *Classification:* The five feature vectors will be stacked together and passed through the ensemble model.
- *Selection and Installation:* The path that had the lowest classification score will be selected and static entries for all sender-receiver IPv4 traffic in both directions will be pushed into the network.

F. Algorithm

Algorithm 1 describes how decisions are made. The first stage of the decision process has a cost of $O(K\,n\,(m + n)\log n)$, where K represents the number of paths considered using Yen's Algorithm on a network consisting of n switches and m links. At this size, the cost will be measured in milliseconds; however, it appears that the probe phase of the decision-making process is what takes up most of the time.

Algorithm 1 Two-stage multimedia route selection

Input: the network topology, the Sender and the Receiver, the number of candidate paths K, the cost weights, and the trained ensemble
Output: the path that the video will use

1: Collect link telemetry from Floodlight: delay, jitter, available bandwidth and loss of every link.
2: Give every link a cost using the composite cost formula (1).
3: Use Yen's algorithm to find the K cheapest paths from the Sender to the Receiver, each at most 4 hops longer than the shortest path.
4: **for** each of these candidate paths **do**
5: Temporarily send the Sender's ping and TCP test traffic over this path.
6: Measure the path with ping and iperf3, and add the link utilisation from the telemetry, giving 11 features.
7: Remove the temporary route.
8: **end for**
9: Give the features of all K paths to the ensemble together, in one call.
10: For each path, take the congestion class with the highest averaged probability (2).
11: Pick the path with the lowest congestion class. If two paths have the same class, pick the one with the lower composite cost; if they are still tied, pick the one that Yen's algorithm listed first.
12: Install the chosen path as static OpenFlow entries for the Sender–Receiver traffic.
13: **return** the chosen path

TABLE III
STRUCTURAL COMPARISON OF THE TOPOLOGIES

Property	GEANT2	Fat-tree $k = 4$
Switches / links	24 / 37	20 / 32
Degree	2–8 (hub s8)	4 core/agg., 2 edge
Diameter (hops)	6	4
Algebraic connectivity	0.31	0.76
Max. betweenness centrality	0.52	0.14
Sender → Receiver	s15 → s23	s12 → s18
Hop-count candidates (hops)	6, 6, 6, 7, 7	4, 4, 4, 4, 6

VI. Implementation and System Design

Software and hardware. The computer system used for this research was a laptop that had an AMD Ryzen 9 8945HS (16-thread) processor and 16 gigabytes of RAM. A virtualized network environment was created using Mininet 2.3.1b4 and Open vSwitch 3.3, which both support OpenFlow 1.3. Floodlight 1.2 (master) was run on Java 8. The software applications used in this research include: Python 3.12, NetworkX, Scikit-Learn 1.9, and XGBoost 3.2. Iperf3 3.16 was also used to generate traffic, while the video streams were created and scored using FFmpeg 6.1. Finally, datasets generated from each exper-

iment were profiled using YData Profiling 4.18.

Topologies. Figure 2 shows the two different types of network topology used in this study; Table III compares the various attributes of these two different topologies. The GEANT2 network has been modeled using an edge list containing 24 nodes and 37 edges that were created for RouteNet [9]. The Fat-Tree network uses a k=4 configuration [10], where there are four core switches, eight aggregation switches and eight edge switches. Each link between any two switches is set up with a speed of 10 Mbits/sec, a delay time of 10 milliseconds per hop and a buffer size of 1000 packets for both networks. Thus, the only factor under examination here is the topology. Additionally, each switch has one host connected to it. However, although all hosts located on the Fat-Tree's core and aggregation switches create and receive background traffic, no such creation/receipt takes place on GEANT2's switches.

Datasets. Each data set was created on an emulated network topology. During each epoch, a new four-class load scenario is applied by the traffic generator; the collector probes all possible sender-receiver pairs along with three randomly selected host pairs using the same 11 feature probe and then assigns each path a label based upon the generator's ground truth bottleneck class (never on a measurement). Since there were limits placed on how many instances could be included in each class (a maximum of one-quarter of the total number of paths), both data sets have equal numbers of paths across all classes. A total of 500 paths (125 per class) were created for the GEANT2 dataset during 65 iterations. In contrast, the Fat Tree dataset initially had 896 paths (224 per class). However, due to seed-based random sampling of 912 rows from the original Fat Tree dataset, there were ultimately 114 iterations produced.

Significance Parameters. k=5; Hop Bound shortest path +4; weights (0.3, 0.1, 0.4, 0.2). Telemetry. Six samples per 2.5s; fifty pings per 50ms; five second iPerf3 TCP; two second drain. Data Split. 70/15/15 stratified seed 40. Random Forest 300 trees. XGBoost with early stopping on the validation split.

Workflow. All modules will run sequentially from one script that trains an ensemble for each topology; evaluates the computational performance offline; evaluates the live performance on both GEANT2 and Fat Tree networks; and creates all figures using the same data that was used to create them.

VII. Results and Discussion

A. Experimental Setup and Evaluation Metrics

A total of five seed-based scenarios were created for each topology (seeds 1000 through 1004). For all links within each scenario, the link's load was generated using the four-class model. Afterward, all links along the hop-count shortest path were set to "Extremely High," which represents the conditions under which load-aware routing would be used. Once the above-mentioned scenarios were established, five different methods were applied sequentially to each scenario and to a single 66-second H.264 video (1920 x 1048 resolution; 60 frames per second; 4,007 total frames; approximately 4.7 Mbit/s):

- *Hop-count:* Shortest Path ($K = 1$)
- *Latency:* Lowest Measured Delay ($K = 1$)
- *Bandwidth:* Widest Path Based Upon Available Bandwidth ($K = 1$)
- *Baseline (Hop-count + ML):* Candidate Strategy for AIRSDN, Five Hop-count Paths Evaluated Using Same Four-Class Ensemble Used for Proposed Method; Only Stage 1 Differs
- *Proposed (Combined + ML):* Composite Top-5 and Ensemble

Metrics included Mean Per-Second Video Throughput, Whole-Stream Video Packet Loss Rate, Mean Per-Second Loss Rate of a 200 kbit/s UDP Probe, Mean Round Trip Time (RTT) of One Ping Per Second, RFC 3550 Jitter [11] of the Probe, FFmpeg Peak Signal-to-Noise Ratio (PSNR) and Structural Similarity Index (SSIM) [12] of Received Video, Ground Truth Class of Chosen Path, and Time Taken at Each Decision Step.

B. Results of the Base Paper / Baseline

The first replication of the base study was performed with an identical configuration to that of the NSFNET scenario described in the original study. A binary logistic regression model was utilized for this replication; and the replication followed the same path as shown in Figure 15 of the original study. The replication of the experiment produced an average RTT of 351 ms, a throughput of 6.52 Mbits/sec, and a PSNR (SSIM) score of 38.1 dB (0.997). These results were very similar to those obtained by the original researchers (354 ms and 37.3 dB). The hop-count routing method produced an average RTT of 1,830 ms and a PSNR score of 20.4 dB. Again, these results were relatively close to those obtained by the original researchers (2,063 ms and 21.5 dB). When comparing the two topologies, a good reference point would be an AIRSDN-style selector with a four-class ensemble. An AIRSDN-style selector with a four-class ensemble is a much better reference point than the binary logistic regression used in the original study.

C. Results of the Proposed Method

Data sets & Feature Correlation. There were no missing data points or duplicate records found while profiling both datasets. Background traffic that is below the capacity limit of a network will result in no queues being formed; thus, RTT should remain relatively constant except at the Extremely High Class level. At the Extremely High Class Level, the median RTT for all other classes were 120.2-120.3 milliseconds on GEANT2 and 80.3-86.5 milliseconds on Fat Tree, but increased to approximately 2.4 seconds. Additionally, ping loss was zero in 84-85 percent of the total number of rows. As illustrated by the

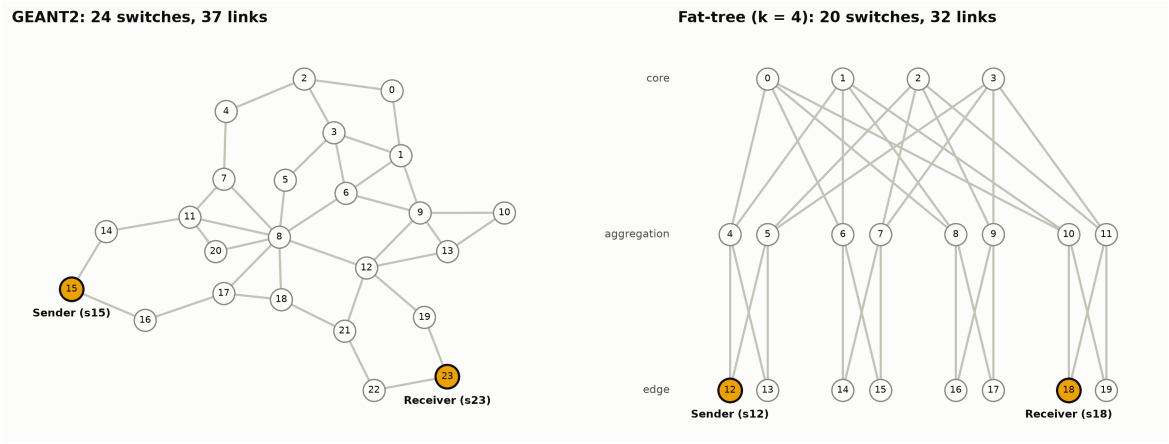


Fig. 2. The two evaluation topologies with the video Sender and Receiver. GEANT2 is meshed with a hub (s8); the fat-tree is layered, with four equal-cost 4-hop paths from s12 to s18, one through each core switch.

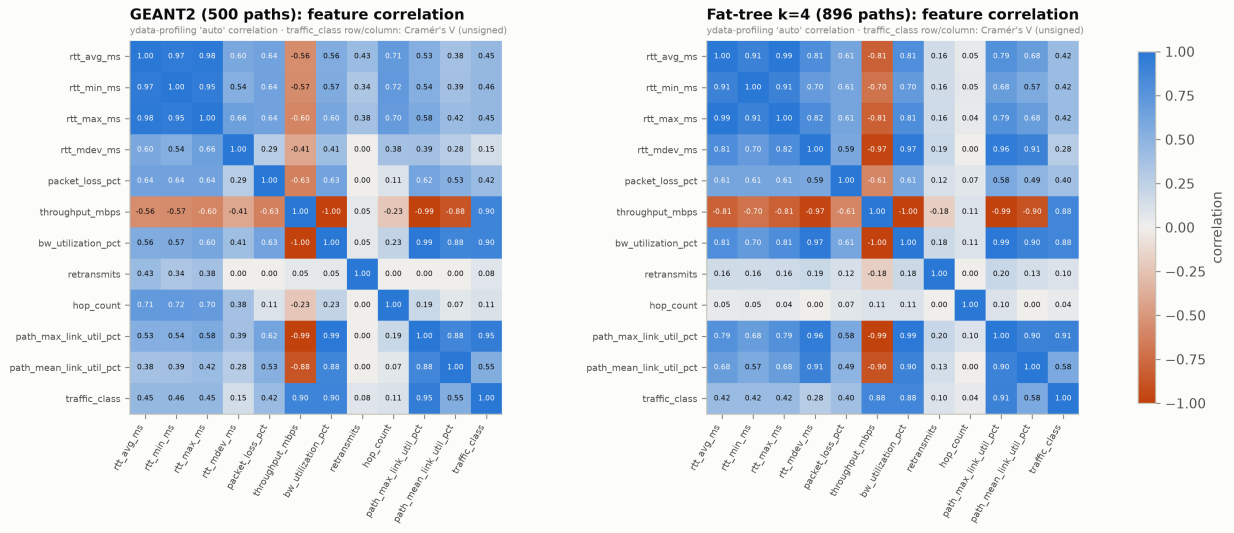


Fig. 3. The feature correlation heat maps from the profiling report ("auto" correlation; the class row/column is calculated using Cramér's V and is unsigned) are shown for both the GEANT2 data set (500 paths) and fat-tree data set (896 paths).

feature correlation heat maps presented in figure 3, there are three main characteristics of these two datasets.

- *Load features carry the class.* Both topological structures show that there was a high correlation between the ground truth `traffic_class` and the three load related feature variables (`throughput_mbps`, `bw_utilization_pct`, `path_max_link_util_pct`) at a level of 0.88–0.95. Since the Cramér's V measure is used for this row, all values will be positive; when the class value goes up so does utilization and down goes throughput.

- *Delay is only moderately informative.* RTT `rtt_avg_ms` has a moderate correlation with class (0.42–0.46) due to the fact that queueing delay remains relatively low prior to a link reaching its maximum capacity at which time it increases rapidly; The non-linear nature of the data is why we were able to define the four types of paths (Section V-C). While RTT does help us distinguish between Extremely High Paths and all other types of paths, it does not allow us to differentiate much between the other three types of paths.

- *Topology affects how much path length matters.* For example, in GEANT2, if you can choose from many different paths that each have different distances, then hop count will be highly correlated with RTT (0.71). However, in the Fat Tree topology, there is almost no correlation between hop count and RTT (0.05) because 75% of the possible paths have exactly 4 hops. Therefore, the Fat Tree model will be unable to rely upon path length for anything.

Classifier accuracy. Table IV shows the results from the tests performed on each ensemble and Figure 4 shows the corresponding confusion matrix for each ensemble. As can be seen in Table IV, both ensembles have achieved a score greater than 95 percent. Specifically, the ensembles classified 73 out of 75 GEANT2 test paths accurately (97.3%) and 133 out of 135 fat-tree test paths accurately (98.5%). In addition, there were very few errors made

TABLE IV
TEST-SET SCORES (MACRO AVERAGES)

	GEANT2 (75 paths)		Fat-tree (135 paths)	
Model	Acc.	F1	Acc.	F1
XGBoost	0.973	0.974	0.985	0.985
Random Forest	0.973	0.974	0.993	0.993
Gradient Boosting	0.973	0.974	0.985	0.985
SVM (RBF)	0.933	0.931	0.926	0.926
Soft-voting ensemble	0.973	0.974	0.985	0.985

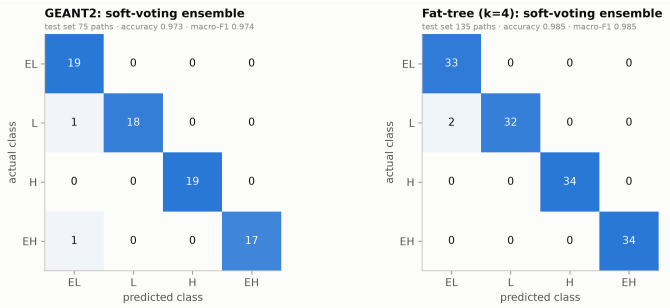


Fig. 4. Confusion matrices of the two ensembles on their test sets (rows: actual class, columns: predicted class).

by either ensemble. Most of these errors occurred when neighboring classes were incorrectly identified.

The first example of misclassification occurred during the testing phase of the fat-tree network. Two low-path tests were incorrectly classified as extremely-low-path tests. Each of these misclassifications was by a single class. A similar misclassification occurred during the testing phase of the GEANT2 network. One low-path test was incorrectly classified as an extremely-low-path test. In addition, one extremely-high-path test was incorrectly classified as an extremely-low-path test. This is the one larger error, and it is the only time that a test path has been misclassified between the low load (EL, L) and high load (H, EH) segments of the classification scale. During real-time operation, the ensembles successfully identified 47 of 50 (GEANT2) and 48 of 50 (fat-tree) candidate paths that were probed.

Routing results. Table V shows the average value for each measure across all five cases. The proposed approach successfully transmitted the video in its entirety in each case on GEANT2 with an average PSNR of 38.43 dB, an average SSIM of 0.998, an average throughput of 4.62 Mbit/s, and an average latency (RTT) of 157 milliseconds. Additionally, in every case the selected route had the lowest amount of congestion possible while still being able to meet the maximum number of hops allowed (see Table VI).

Figure 5 illustrates this issue. For GEANT2 Scenario 0, the forced congestion is along the default hop count route s15-s14-s11-s8-s12-s19-s23. The remaining short paths also go over a high link (s17-s18).

Thus, each of the five possible hop count routes has either a high or extremely high bottleneck. Hop count routing will use the forced extremely high route. Latency routing and the baseline (whose candidate routes were limited to those same short paths), however, used a 6-hop high route. While the baseline selected the optimal route among its options, no option was lightly loaded. A composite cost penalty was applied to the heavily loaded links. It provided a 7-hop detour route s15-s16-s17-s8-s18-s21-s22-s23 which went around the bottleneck area via links that were all extremely low. Both the ensemble and widest-path routing identified this detour as being available and selected it. This occurred again in Scenario 3. Scenarios 1, 2, and 4 had their candidate routes containing the optimal route prior to selection by both methods. As shown in Figure 6 (left), there were consequences for video quality based upon the route taken. When using a 6-hop high route, the baseline and latency routing maintained an RTT of approximately 1.3 seconds during much of the transfer time due to filling of queues. Meanwhile, hop count routing on the forced extremely high route lost nearly all echoes resulting in an RTT greater than 10 seconds. Due to avoiding congested links, the proposed methodology kept the RTT close to 150 milliseconds throughout the entire transfer process without experiencing the spikes experienced by other methodologies.

D. Comparative Analysis

Figure 7 illustrates the performance of all five methods on both networks, whereas Figure 8 illustrates how each method made its decision.

GEANT2: The proposed method reduced the average video loss by 2.1% (from 9.1% in Scenario 0 and 1.5% in Scenario 3) compared to the baseline; increased the average PSNR from 31.4 dB to 38.4 dB; increased the average SSIM from 0.950 to 0.998; decreased the average RTT from 532 ms to 157 ms; and had an average RTT of 13,064 ms for Hop Count Routing. Therefore, this method has eliminated the high latency associated with the congested links which resulted in no additional frame losses (video loss = 0.0%) and maintained a high level of video quality (PSNR = 38.4 dB; SSIM = 0.998). Latency-Based Routing performed identically to the Baseline since it used the exact same 6-hop High Path as did the Baseline. Bandwidth-Based Routing produced identical results to those obtained using the Proposed Method regarding each of the three quality metrics. Since the path selected by Bandwidth-Based Routing will always be the widest possible path available, its most loaded (bottleneck) link will also be the least loaded of any available path. On GEANT2, Bandwidth-Based Routing chose the same detour as did the Proposed Method. Hop Count-Based Routing experienced significant degradation in performance when forced onto the Extremely High Path due to losing 82.6% of the video (resulting in a PSNR value of 14.8 dB).

Fat-Tree: As shown in Table VI, all four load-aware methods were able to achieve the highest possible classification level in each scenario; therefore, all four methods

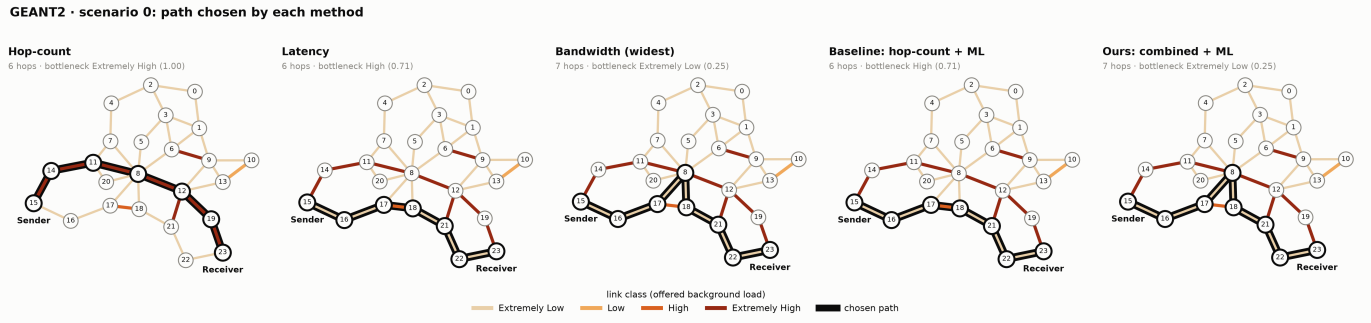


Fig. 5. Paths chosen in GEANT2 scenario 0, drawn over the offered-load class of every link. Only bandwidth routing and the proposed method leave the High and Extremely High region.

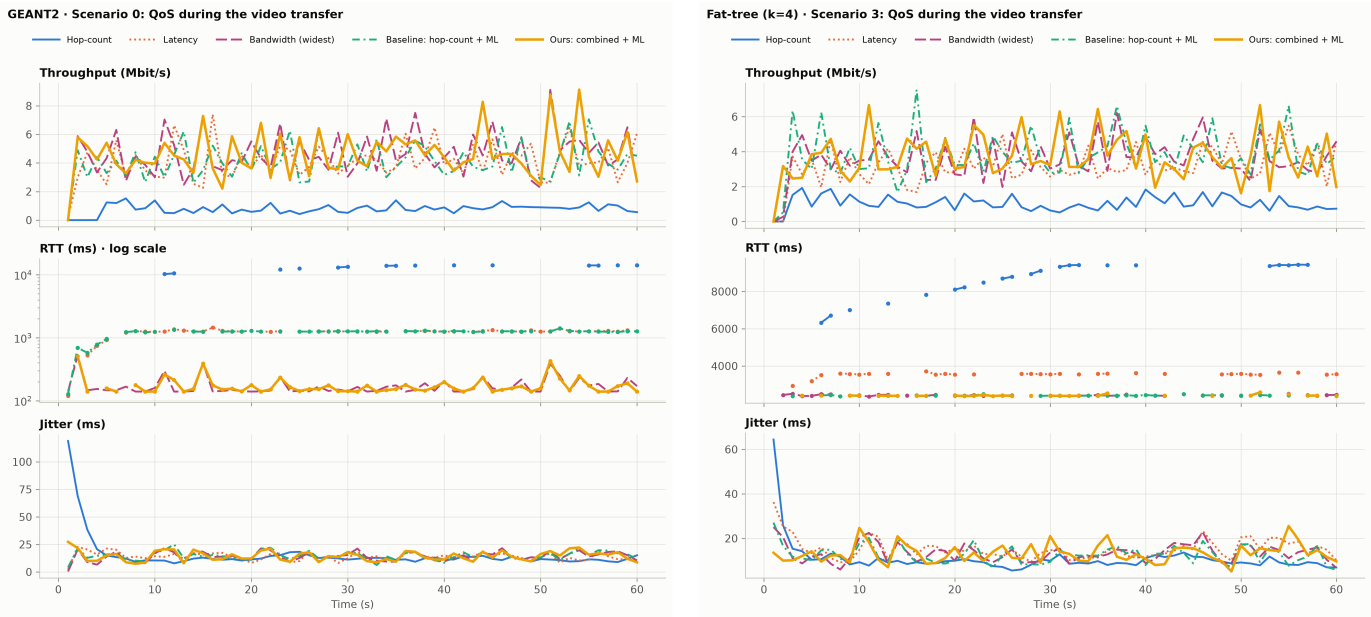


Fig. 6. Per-second video throughput, RTT (log scale) and jitter. Left: GEANT2 scenario 0. Right: fat-tree scenario 3, where every path within the hop bound crosses an Extremely High link.

had identical average classifications (1.40) and very similar performance metrics: 5.9–8.2% loss and 29.2–29.5 dB. Due to the Fat-Tree's Equal-Cost Multipath Structure, there are very few options left for alternative routes; those remaining are already some of the shortest paths that all load-aware methods consider. When using the forced path S12-S4-S0-S10-S18, this will disconnect one of each endpoint's two Up Links and thus eliminate two of the four Core Paths. This leaves only two paths through nodes S5 and S11. Both of these paths crossed an Extremely High Link in Scenarios 0 and 3, causing every method to lose 11–27% of the Video and no Routing Choice could prevent this loss (see Fig. 6 Right). In Scenarios 1, 2, and 4 every Load-Aware Method was able to deliver the Video without any Loss. The minor variations observed among the various Methods are due to Run-To-Run Variability rather than Different Decisions Made by Each Method. For example, in Scenario 3 the Baseline Method and Proposed Method selected the Exact Same Path and Lost 16.4% and 20.5% of the Video in Two Consecutive Runs.

Overhead. Figure 9 illustrates how GEANT2's decision-making process is broken down into individual components. As hop-count routing does not require either telemetry or probes; 2.2 ms was used for path calculation and 434 ms were required to send the calculated route via Floodlight's REST API. The two machine learning techniques evaluated here both required approximately 19–20 seconds for telemetry collection.

However, the proposed technique collected data for 128 seconds per decision compared to 217 seconds for the baseline technique due to the fact that the candidate paths selected by the proposed technique had significantly lower levels of congestion. A probe of a congested path will wait until both ping and iperf3 time out (iperf3 145 seconds vs. 70 seconds).

In addition to collecting data, the total amount of time spent on computing and installing routes was 722 milliseconds (proposed) versus 764 milliseconds (baseline). Of this total time, approximately half was devoted to installing the route. Offline, it took between 1.1–2.2 milliseconds to

TABLE V

MEAN RESULTS OVER FIVE SCENARIOS PER TOPOLOGY (WHOLE-STREAM VIDEO LOSS; RTT N/A WHERE EVERY ECHO WAS LOST)

	GEANT2							Fat-tree ($k = 4$)						
Method	Thr. Mbit/s	Loss %	RTT ms	Jitter ms	PSNR dB	SSIM	Class 0–3	Thr. Mbit/s	Loss %	RTT ms	Jitter ms	PSNR dB	SSIM	Class 0–3
Hop-count	0.76	82.6	13,064	17.0	14.8	0.739	3.00	0.96	78.3	8,607	10.4	14.9	0.746	3.00
Latency	4.50	2.3	530	14.2	31.3	0.949	1.00	4.21	8.2	1,019	13.9	29.4	0.902	1.40
Bandwidth	4.61	0.0	169	13.3	38.4	0.998	0.20	4.30	6.6	798	13.9	29.4	0.901	1.40
Baseline (hop + ML)	4.50	2.1	532	14.3	31.4	0.950	1.00	4.33	5.9	793	13.2	29.5	0.903	1.40
Proposed	4.62	0.0	157	13.2	38.4	0.998	0.20	4.31	6.6	792	13.3	29.2	0.893	1.40

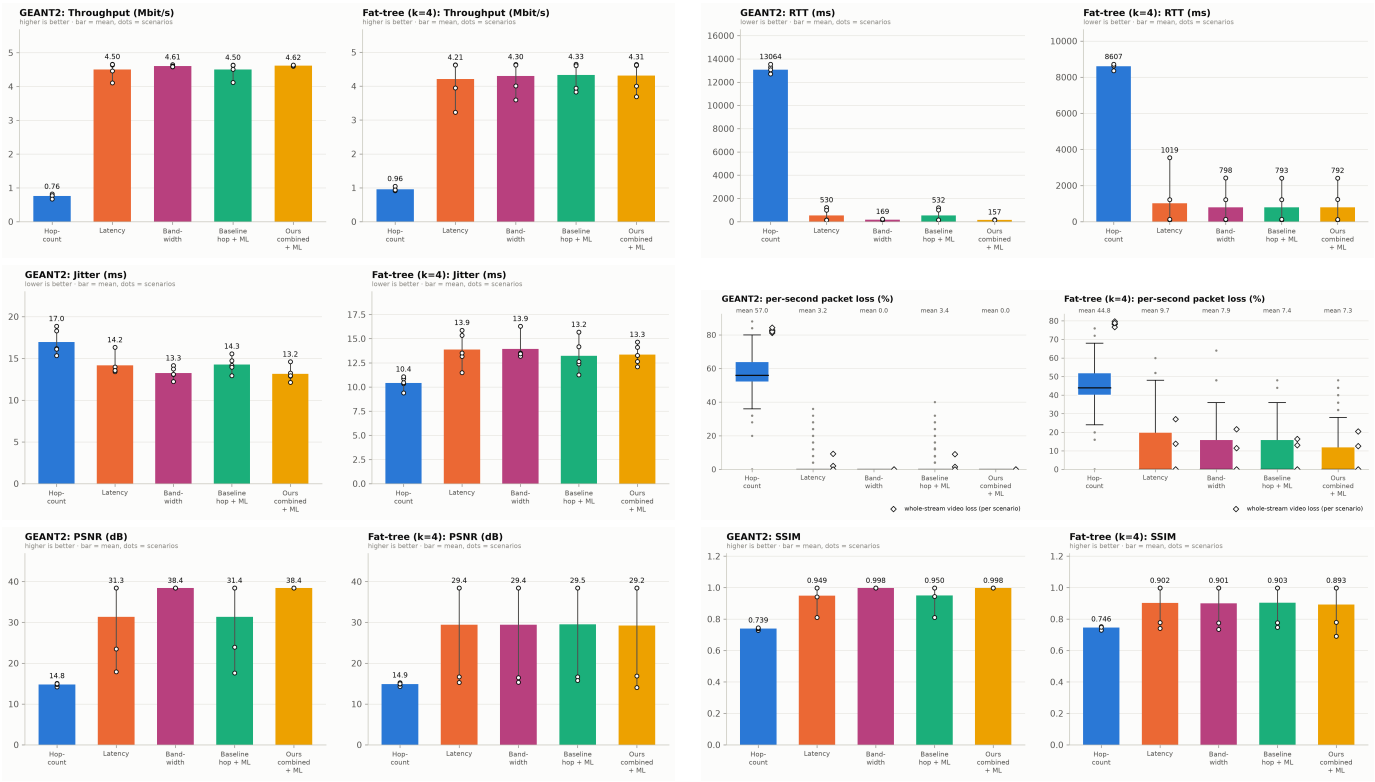


Fig. 7. QoS and video quality of the five methods on GEANT2 and the fat-tree. Throughput, RTT, jitter, PSNR and SSIM: bars are means, dots are scenarios. Packet loss: per-second probe loss pooled over scenarios (diamonds: whole-stream video loss).

TABLE VI

GROUND-TRUTH CLASS OF THE CHOSEN PATH PER SCENARIO, AND THE BEST CLASS REACHABLE WITHIN THE HOP BOUND

	GEANT2					Fat-tree				
Scenario	0	1	2	3	4	0	1	2	3	4
Best reachable	0	1	0	0	0	3	0	0	3	1
Hop-count	3	3	3	3	3	3	3	3	3	3
Latency	2	1	0	2	0	3	0	0	3	1
Bandwidth	0	1	0	0	0	3	0	0	3	1
Baseline	2	1	0	2	0	3	0	0	3	1
Proposed	0	1	0	0	0	3	0	0	3	1

complete stage 1 for k = 5 using any metric. Inference for stage 2 is completed in one batched call (figure 10): the feature vectors from every candidate are stacked into a single batch and then passed into the ensemble.

The off-line runtimes were 15.8 ms for k=5 and 15.1 ms for k=10. When the ensemble was run sequentially (one after another) the same operations took 75.1 ms for k=5 and 157.7 ms for k=10. Therefore, batching decreased the execution time of the model by 4.8 times at k=5 and 10.4 times at k=10.

Additionally, the computational cost stays almost constant as k increases, since each member model has a fixed overhead cost per invocation which is only incurred once per decision rather than once per candidate.

E. Discussion of Findings and Limitations

Objectives.

Objective 1 was achieved: The proposed method selected the least congested possible route for each of the ten scenarios tested. It also successfully selected the least congested possible route in the two scenarios tested on GEANT2 where the Baseline Method failed.

Objective 2 was achieved at 97.3% and 98.5% ensemble accuracy.

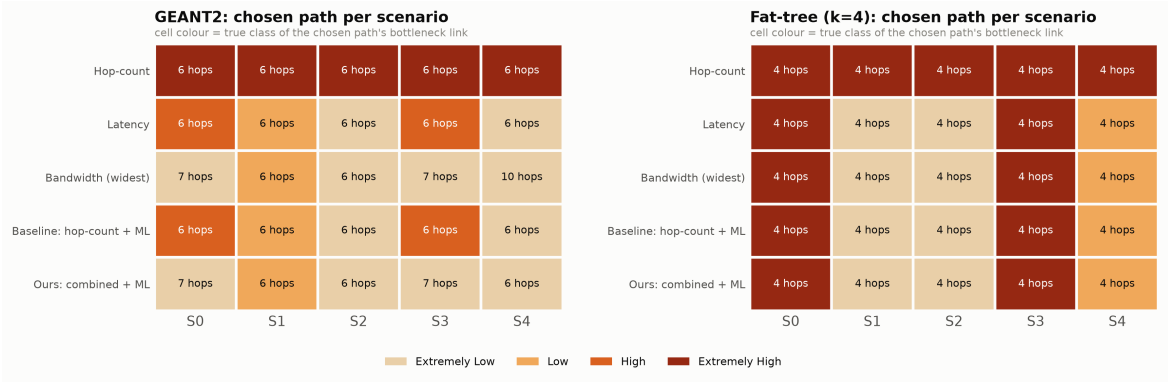


Fig. 8. True class and length of the path each method chose in every scenario.

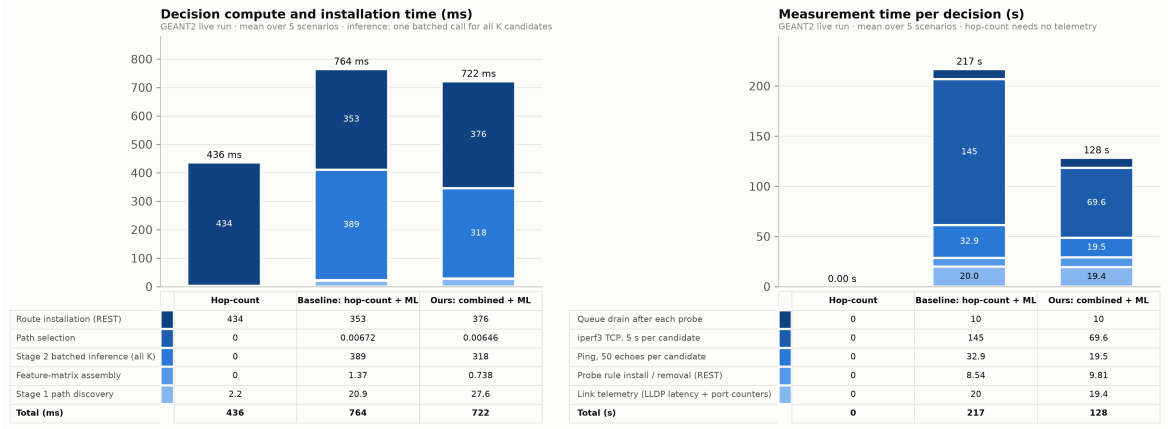


Fig. 9. Average per-step decision overhead on GEANT2 for five different scenarios. On the left side is the time spent on computation and route installation (in milliseconds); on the right side is the time spent on measurements (in seconds). All steps are listed in the table, even if they were too small to be seen.

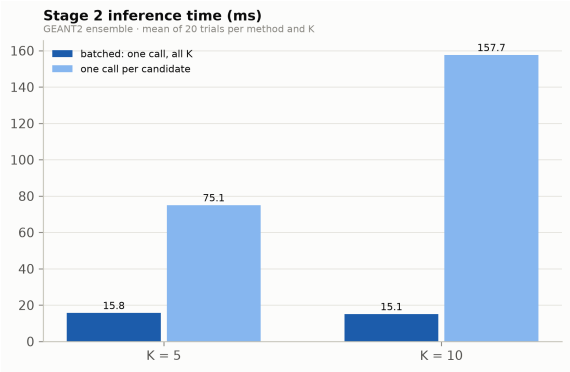


Fig. 10. Stage 2 inference time on GEANT2 (offline, 20 trials per Stage 1 method): one batched call for all candidates against one call per candidate.

Objective 3 was achieved on both networks. Improvement was evident on GEANT2 but not on Fat-Tree.

Objective 4 was achieved: Each component of the algorithm was timed and inference was performed using batches.

How the improvement occurred. Improvement occurred only if the optimal solution lay outside the Hop Count Candidate Set. Due to its topology, GEANT2 provides many opportunities to avoid areas of high traffic volume near the shortest path(s) through a slight increase in distance via a detour. The Composite Cost identifies such an opportunity. Fat-Trees have two uplinks per edge switch. Once congestion occurs on one of these uplinks, there are very limited options available to find an alternative route. Therefore, the Hop Count Candidates will typically contain the optimal solution. Further testing should be conducted on Fat-Trees with greater than two uplinks ($k > 4$), or Fat-Trees with congestion placed on Core Links.

Bandwidth routing as a strong baseline. In both topologies, widest-path routing produced results identical to those obtained with the proposed approach; there was no requirement for probing. A path's quality is determined by the link carrying the greatest amount of traffic; therefore, the widest path will minimize this directly. The ML phase provides additional benefit when telemetry data is incorrect or outdated, something that was not tested in either of the two controlled environments described above. Additionally, the ML phase produces a grade-based classification that cannot be achieved using a single metric.

Limitations. (I) The results were obtained using five different scenarios for each type of network topology with an emulated bandwidth of 10 Mbits/s; therefore, minor variations in performance should be expected when com-

paring two runs of the same experiment – particularly on the Fat-Tree topology due to the inherent variability associated with running experiments. (II) Since all of the methods were executed sequentially on the same set of data, there was some drift in the amount of background traffic during execution time. (III) Live Inference required 318–389 milliseconds compared to 15 milliseconds for Offline Inference. This difference appears to be primarily related to the fact that both the Emulation thread, Video thread and Model thread were sharing a single CPU. (IV) Probing makes a single routing decision take several minutes. Therefore, this method is best suited for supporting Long-Lived Flows. (V) Every topology requires a unique dataset and model.

VIII. Conclusion and Future Work

The primary objective of this research was to investigate the use of hop count based candidate selection and binary labeling for multimedia routing in SDN by AIRSDN. In order to achieve this, we developed an algorithm which uses a composite cost top five candidate search and a four class soft voting ensemble to evaluate each candidate in one batched evaluation. We then evaluated our proposed method using actual H.264 video on both GEANT2 and a K=4 fat tree. On GEANT2, our approach identified the best possible path in every instance and successfully transmitted the video with zero loss; resulting in a PSNR value of 38.4 dB and a round-trip time (RTT) of 157 ms. Our approach outperformed the Hop Count Candidate Baseline Method, which had a 2.1% loss rate, a PSNR value of 31.4 dB, and an RTT of 532 ms. Furthermore, our approach took 41% less time per decision. Due to the limited number of alternatives available on the fat tree (due to the sparsity of its uplink connections), all load-aware methods were able to identify the same optimal path and thus yielded comparable results. One of the primary limitations of our current implementation is the amount of time needed for probing, the small number of scenarios used in testing, and the necessity to train the model independently for each topology. Future research may consider substituting full probing with passive telemetry for lightly loaded candidates and/or studying larger fat trees and dynamic traffic patterns that occur during a flow's lifetime.

References

[1] A. D. İpek, M. Cicioğlu and A. Çalhan, "AIRSDN: AI based routing in software-defined networks for multimedia traffic transmission," *Computer Communications*, vol. 240, p. 108222, 2025.

[2] E. Ç. Kük and M. Erel-Özçevik, "SD-CDN: Software defined content delivery network for QoE optimization in mobile networks," *Computer Networks*, vol. 205, p. 108686, 2022.

[3] C. Cui and H. Xu, "A path selection strategy for cloud computing traffic optimization based on neural network traffic prediction," *International Journal of Grid and Distributed Computing*, vol. 9, no. 1, pp. 25–36, 2016.

[4] P. Bakonyi, T. Boros and I. Moldován, "Classification based load balancing in content delivery networks," IEEE, 2017.

[5] M. Babayigit and A. Ulu, "Deep learning for load balancing of SDN-based data center networks," *International Journal of Communication Systems*, vol. 34, no. 7, p. e4760, 2021.

[6] A. A. Al-Mimi et al., "Artificial intelligence-based load balancing schemes in software-defined networks: A survey," *Computers, Materials & Continua*, vol. 70, no. 1, pp. 251–270, 2022.

[7] M. H. H. Mohamed et al., "A new approach for server-based load balancing using software-defined networking," in *Proc. 8th IEEE Int. Conf. Intelligent Computing and Information Systems (ICICIS)*, 2017.

[8] J. Y. Yen, "Finding the K shortest loopless paths in a network," *Management Science*, vol. 17, no. 11, pp. 712–716, 1971.

[9] K. Rusek, J. Suárez-Varela, P. Almasan, P. Barlet-Ros and A. Cabellos-Aparicio, "RouteNet: Leveraging graph neural networks for network modeling and optimization in SDN," *IEEE Journal on Selected Areas in Communications*, vol. 38, no. 10, pp. 2260–2270, 2020.

[10] M. Al-Fares, A. Loukissas and A. Vahdat, "A scalable, commodity data center network architecture," in *Proc. ACM SIGCOMM*, 2008, pp. 63–74.

[11] H. Schulzrinne, S. Casner, R. Frederick and V. Jacobson, "RTP: A transport protocol for real-time applications," IETF, RFC 3550, 2003.

[12] Z. Wang, A. C. Bovik, H. R. Sheikh and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.